

Data Structures – Sessional Exam

Past-Paper Analysis: Topic Weightage, Priority List & Question Patterns

0. Scope & Method

Sessional scope (from your syllabus slides): Unit 1 Introduction & Algorithm Analysis (incl. linear search) → Unit 2 Arrays (address calculation, polynomial, sparse matrix) → Unit 3 Stack (polish notation, infix/postfix/prefix, recursion) → Unit 4 Queue (simple, circular, priority) → Unit 5 Linked Lists (singly, circular, doubly). Sorting algorithms, binary search, trees, graphs, B-trees and MST are **not** in your sessional syllabus, so those questions were excluded.

Papers analysed (4): Jan-2022 (50 marks, 2 hrs), Jan-2023, Jan-2025 and Dec-2025 (100 marks, 3 hrs each). Frequency = number of papers (out of 4) in which the topic appeared. Because the sessional paper is set separately, treat this as likelihood, not a guarantee.

Paper-wise share of sessional topics

Paper	Max marks	In-scope marks (core)	Share of paper	Borderline (linear search half only)	Questions in scope
Jan-2022	50	18	36%	+7 (Q2B)	Q1A, Q1B, Q1C (OR)
Jan-2023	100	46	46%	—	Q1B, Q2A, Q2B, Q3A, Q3B, Q6B
Jan-2025	100	34	34%	—	Q1A, Q1B, Q1C, Q2A, Q2B, Q2C
Dec-2025	100	34	34%	+4 (Q3A)	Q1A, Q1B, Q1C, Q2A, Q2B, Q2C

Key observation: in both 2025 papers, **every in-scope question sits in Q1 and Q2 (34 marks)** – Q3 onwards moves to searching/sorting, trees and graphs.

1. Topic Weightage Table

1A. Unit-level weightage (unique in-scope marks across all 4 papers = 132)

Unit	Papers with ≥1 question	Marks across 4 papers	Share of in-scope marks	Typical marks per appearance
Unit 5 – Linked List (incl. polynomial questions)	4/4	34	25.8%	6 (one 10-mark circular-LL question)
Unit 3 – Stack (infix/postfix conversion & evaluation)	4/4	32	24.2%	6–8
Unit 2 – Arrays (address calculation)	4/4	24	18.2%	6
Unit 4 – Queue (circular, simple/static, priority)	4/4*	22	16.7%	6–10
Unit 1 – Intro & Analysis (linear vs non-linear, complexity)	3/4	14	10.6%	4–6
OR-slot: Priority queue (2-D array) / Doubly LL end-insert	1/4	6	4.5%	6

*Queue appears in all 4 papers when the Jan-22 OR option (priority queue) is counted. Polynomial questions sit under Arrays in the syllabus (slide 46) but are answered with a linked list, so they are counted under Linked List.

1B. Topic-level weightage

Unit	Specific topic	Frequency (of 4)	Appeared in	Typical marks	Priority	Notes
Unit 1 – Intro & Analysis	Linear vs non-linear data structures (difference, examples, real-world applications)	2/4 (50%)	Jan-23 Q6B; Jan-25 Q1A	4–6	High	Short theory; Jan-23 also asked for two real-world applications.
Unit 1 – Intro & Analysis	Algorithm complexity; best / average / worst case	1/4 (25%)	Dec-25 Q1A	4	Medium	Linear search is the syllabus's worked example (slides 21–30).
Unit 1 – Intro & Analysis	Linear search + time-complexity comparison with binary search	2/4 (50%)	Dec-25 Q3A; Jan-22 Q2B	4–7	Low–Med	Borderline: only linear search is in your slides; binary search is not.
Unit 2 – Arrays	2-D array address calculation (row-major AND column-major)	4/4 (100%)	Dec-25 Q1B; Jan-22 Q1A; Jan-23 Q1B; Jan-25 Q1B	6	Very High	Identical 6-mark numerical every single paper.
Unit 2 – Arrays	Polynomial representation with singly linked list + node format + algorithm	2/4 (50%)	Dec-25 Q1C; Jan-25 Q1C	6	High	Algorithm asked: count terms (Dec-25), display (Jan-25).
Unit 2 – Arrays	Sparse matrix representation	0/4	—	—	Low	In syllabus (slides 47–50) but never asked.
Unit 3 – Stack	Infix → postfix conversion (stack table)	3/4 (75%)	Dec-25 Q2A; Jan-23 Q2B; Jan-25 Q2A	6–8	Very High	Short expressions with brackets; ^ appears in Jan-23.
Unit 3 – Stack	Postfix evaluation (stack status after every step)	4/4 (100%)	Dec-25 Q2A; Jan-22 Q1B; Jan-23 Q3B; Jan-25 Q2A	6	Very High	Standalone (2022, 2023) or combined with conversion (2025).
Unit 3 – Stack	PUSH/POP/PEEP algorithms, RECOGNIZE, prefix, recursion	0/4	—	—	Low	Never asked; study last.
Unit 4 – Queue	Circular queue – trace front/rear OR justify over simple queue	2/4 (50%)	Jan-23 Q2A (10); Jan-25 Q2B (6)	6–10	High	Jan-23: 10 operations on a size-6 queue, show front & rear each step.
Unit 4 – Queue	Simple/static-array queue algorithms and application design	2/4 (50%)	Dec-25 Q2C; Jan-25 Q2B	6	High	Dec-25: add patient, call next, view waiting list.
Unit 4 – Queue	Priority queue using 2-D array	1/4 (25%)	Jan-22 Q1C (OR option)	6	Medium	Was an either/or choice with doubly LL.
Unit 5 – Linked List	Singly LL algorithms (insert front/end, delete last, count, display)	2/4 (50%)	Dec-25 Q2B; Jan-25 Q2C	6	High	Counting polynomial questions, LL appears in 4/4 papers.

Unit	Specific topic	Frequency (of 4)	Appeared in	Typical marks	Priority	Notes
Unit 5 – Linked List	Circular LL – insert/delete at start, end, any position	1/4 (25%)	Jan-23 Q3A	10	Medium	Largest single algorithm question (slides 144–156).
Unit 5 – Linked List	Doubly LL – insert node at end	1/4 (25%)	Jan-22 Q1C (OR option)	6	Medium	Slides 157–166.
Cross-cutting (Units 4 & 5)	Static vs dynamic allocation – justify statement + write algorithm	2/4 (50%)	Dec-25 Q2B, Q2C; Jan-25 Q2B	6	High	Newest papers; higher Bloom levels (BL5 justify, BL6 design).

2. High-Yield Priority List

Study in this order. Together, topics 1–4 account for **about 90% of the in-scope marks** in past papers.

Rank	Topic to master	Evidence from past papers	What you must be able to do	Practice target
1	Infix → postfix conversion and postfix evaluation	Appears in 4/4 papers; 32 marks over 4 papers; 6–8 marks each time.	Build the symbol stack output table without errors; handle brackets, left-to-right associativity and ^. Evaluate by pushing operands and popping B then A for each operator.	Do 10 expressions; the Appendix A answers for the 5 past expressions let you self-check.
2	Array address calculation (row-major & column-major)	4/4 papers; 6 marks each; same format every time.	Write both formulas from memory; count rows/columns from the lower/upper bounds (LB = 0 or 1); compute both orders for the requested element.	Redo the 5 past problems and then 5 new ones with different bounds.
3	Singly linked list algorithms + polynomial representation	Linked list in 4/4 papers (34 marks). Polynomial in both 2025 papers.	Node format (coefficient, exponents, link); algorithms for insert front / end, delete last, count nodes, display; dynamic allocation with malloc.	Write each algorithm in step-form (as in slides 124–143) until you can do it without looking.
4	Queue: circular queue trace + static array queue algorithms	Queue in 4/4 papers; 10-mark trace in Jan-23; design/justify questions in both 2025 papers.	Show front and rear after each operation; explain why circular beats simple queue in static memory; write insert/delete algorithms; apply a queue to a scenario (e.g., patient registration).	Trace the Jan-23 sequence, then invent 2 new sequences.
5	Unit 1 theory: linear vs non-linear DS, best/average/worst case	3/4 papers; 4–6 marks; easy marks.	A 5–6 point comparison table with examples (stack, queue vs tree, graph) and 2 real-world applications; define the three cases using linear search.	Prepare one ready-made answer for each.
Next	Circular and doubly linked lists; priority queue	One appearance each, but they are the longest algorithm questions (10 marks in Jan-23).	Insert/delete at start, end, any position for circular LL; insert at end for doubly LL; priority queue via 2-D array.	Cover after topics 1–5 are solid.
Low	Sparse matrix, PUSH/POP/PEEP, RECOGNIZE, prefix, recursion	Never asked in 4 papers.	Skim only; your sessional is set separately, so don't skip entirely if time allows.	Last-day revision.

3. Question Patterns

3A. Recurring question formats

#	Pattern	What the question looks like	Evidence	How to prepare
1	Address-calculation numerical	Given array dimensions, base address, element size and lower bound: find the address of one element in row-major and column-major order. Always 6 marks.	4/4 papers. Lower bound 0 (Dec-25) or 1 (Jan-22, Jan-23, Jan-25); Jan-23 asked two different elements, one per order.	Memorise the two formulas; always state the assumptions (lower bound, rows/columns).
2	Convert + evaluate with a stack	Convert an infix expression to postfix, then evaluate the result using a stack, showing every step.	Combined in Dec-25 and Jan-25 (6 marks); separate in Jan-22 and Jan-23 (6–8 marks each).	Practise the two steps back-to-back on one expression.
3	“Write an algorithm to ...” on lists and queues	Step-by-step algorithm for a single operation (count nodes, display, insert at end, insert/delete at front or last, priority queue, circular list).	8 algorithm questions across the 4 papers (3 in Dec-25, 3 in Jan-25, 1 in Jan-23, 1 in Jan-22). 6 marks each; 10 for Jan-23 circular LL.	Learn the structure: check empty → create node → link → update pointers; state input/output.
4	Polynomial with singly linked list	“Represent the polynomial using a singly linked list, explain the node format, write an algorithm to ...”. The polynomial has 2–3 variables.	Identical wording in Dec-25 and Jan-25; only the final algorithm changed (count terms / display).	Learn the node layout and draw the list. Likely variants: add two terms, search a term, evaluate (prediction).
5	Justify + algorithm (higher Bloom level)	A statement is given (“dynamic list is flexible”, “circular queue is better under static allocation”); justify it with examples and write an algorithm.	Dec-25 Q2B, Jan-25 Q2B (BL5, 6 marks).	Prepare a small worked example (memory waste in a simple queue; growing a linked list with malloc).
6	Design a system with the right data structure	A real-world scenario; choose the structure, state allocation type and formulate algorithms.	Dec-25 Q2C patient registration (queue, static memory); Dec-25 Q2B customer list.	Map scenarios: waiting lines → queue, growing records → linked list, undo/expressions → stack.
7	Trace / show status after each step	Show front and rear (circular queue) or stack contents after each operation.	Jan-23 Q2A (10 marks); Jan-22 Q1B; Jan-23 Q3B.	Draw a table with one row per operation; never skip steps.
8	Differentiate (theory)	Differentiate two concepts with examples.	Linear vs non-linear: Jan-23 Q6B and Jan-25 Q1A.	Use a two-column table with 5–6 points.

3B. Blueprint of the two newest papers (Q1 and Q2 are identical in structure)

Slot	Marks / Bloom	Dec-2025	Jan-2025	Topic type
Q1A	4 / BL2–4	Algorithm complexity; best, average, worst case	Linear vs non-linear data structures	Unit 1 theory
Q1B	6 / BL3	4×4 array, base 4000, 4-byte elements, Ary[3][3]; lower bound 0	4×5 array, base 2500, 4-byte elements, A[3][4]; lower bound 1	Array address numerical

Slot	Marks / Bloom	Dec-2025	Jan-2025	Topic type
Q1C	6 / BL3	Polynomial $\rightarrow$ singly LL; node format; algorithm to count terms	Polynomial $\rightarrow$ singly LL; node format; algorithm to display	Linked list + polynomial
Q2A	6 / BL3	$2 + 3 * (9 - 4) / 5$: infix $\rightarrow$ postfix, evaluate	$5 + 3 - (4 - 2) * 9 / 6$: infix $\rightarrow$ postfix, evaluate	Stack
Q2B	6 / BL5	Justify “dynamic list is flexible”; algorithm to add a customer at end	Justify “circular queue better under static allocation”; algorithm to insert in simple queue	Justify + algorithm
Q2C	6 / BL6	Patient registration system (static memory): add, call next, view waiting list	Algorithms: insert node at front, delete node from last (singly LL)	Design / formulate algorithms

If your sessional follows this house style, expect roughly: **one short theory question, one array-address numerical, one stack conversion/evaluation, one linked-list algorithm, one queue question.**

3C. Patterns that have NOT appeared

- Time-complexity questions on sorting are out of your scope; in scope, only the best/average/worst-case definition (Dec-25) and the linear-vs-binary search comparison (Jan-22, Dec-25) have been asked.
- The syllabus ends with a time-complexity summary for array, queue, stack and linked list (slides 167–170). It has never been asked, but it is a cheap 4-mark risk: keep a one-page revision table.
- Sparse matrix, recursion, prefix, PUSH/POP/PEEP and the RECOGNIZE algorithm have not been asked in 4 papers.

Appendix A – Answer key for repeated numericals (verified by calculation)

Use this to check your own working. Formulas: Row-major = $\text{Base} + w \times [(i - \text{LBr}) \times \text{Columns} + (j - \text{LBC})]$; Column-major = $\text{Base} + w \times [(j - \text{LBC}) \times \text{Rows} + (i - \text{LBr})]$.

Paper / Q	Given	Answer
Dec-25 Q1B	Ary[4][4], base 4000, w = 4, LB = 0, element Ary[3][3]	Row-major = 4060; Column-major = 4060 (square array, same position)
Jan-22 Q1A	arr[1..10][1..15], base 200, w = 2, element arr[8][9]	Column-major = 374; Row-major = 426
Jan-23 Q1B	arr[1..30][1..10], base 400, w = 4	arr[28][5] row-major = 1496; arr[16][7] column-major = 1180
Jan-25 Q1B	A[4][5], base 2500, w = 4, LB = 1, element A[3][4]	Row-major = 2552; Column-major = 2556

Paper / Q	Infix / postfix given	Postfix form	Result
Dec-25 Q2A	$2 + 3 * (9 - 4) / 5$	$2\ 3\ 9\ 4 - * 5 / +$	5
Jan-22 Q1B	M: $8\ 2\ 3 \wedge / 2\ 3 * + 5\ 1 * -$	(already postfix)	2
Jan-23 Q2B	$8 + 4 / 2 * 3 - 6 + 7 - 8 \wedge 1$	$8\ 4\ 2 / 3 * + 6 - 7 + 8\ 1 \wedge -$	7

Paper / Q	Infix / postfix given	Postfix form	Result
Jan-23 Q3B	8 7 6 3 / 4 2 // + /	(already postfix)	1
Jan-25 Q2A	5 + 3 – (4 – 2) * 9 / 6	5 3 + 4 2 – 9 * 6 / –	5

Appendix B – One-page revision cheat sheet

Topic	Must-remember points
Postfix conversion	Operand → output. “(” → push. “)” → pop until “(”. Operator → pop while top has higher or equal precedence (for ^, equal precedence is NOT popped, it is right-associative); then push. At the end pop all.
Postfix evaluation	Operand → push. Operator → pop B (top), pop A, compute A op B, push result. Final stack value is the answer.
Circular queue	Insert: rear = (rear + 1) mod N; Delete: front = (front + 1) mod N. Follow the convention used in your lecture slides (CQINSERT / CQDELETE, slides 108–111). Reason it beats a simple queue: freed front slots are reused, so no false “overflow”.
Singly LL node	INFO LINK (next). Polynomial node: coefficient power of X power of Y power of Z link. Counting terms = counting nodes (COUNT_NODES, slide 140).
Dynamic vs static	Static (array): fixed size decided at compile time, wasted or insufficient space. Dynamic (malloc + linked nodes): grows and shrinks at run time, one node at a time.
Complexity cases	Best = least work (linear search: element is first, $O(1)$); worst = most work (last or absent, $O(n)$); average = expected over all positions (about $n/2$ comparisons, $O(n)$).
Linear vs non-linear	Linear: elements in a sequence, single level (array, stack, queue, linked list). Non-linear: hierarchical or network relationships (tree, graph).